

EVOLUTION AI 算法模型

把 `ai-car-styling` Skill 完整展开为一套**独立可运行的算法模型**。
不依赖 Skill 调用层，直接用 Python 代码调用，可复用到任何工程项目。

模块构成

```
algorithm_model/
├─ README.md          # 本文档
├─ requirements.txt    # 依赖
├─ main.py            # 一站式 CLI 入口（7 个子命令）
├─ api.py             # 统一对外 7 大 API
├─ test_all.py        # 一站式自检脚本（5 模块）
├─
├─ car_modeling/      # 模块 1：参数化车身建模
│   ├─ car_params.py  # 22 维 CarParams + 边界校验
│   ├─ body.py        # 主车身截面壳体（process 开关）
│   ├─ body_nurbs.py  # NURBS 车身（STEP 导出）
│   ├─ body_ends*.py  # 车身端部 4 种连续性方案
│   ├─ accessories_nurbs.py # NURBS 附件
│   ├─ blending.py    # 混接曲面
│   ├─ continuity_checker.py # G0/G1 连续性检查（0.1mm/1°）
│   ├─ parametrize.py # 参数化工具
│   ├─ trim.py / sop_checklist.py
│   ├─ glass.py       # 玻璃（前/后挡风 + 天窗 + 侧窗）
│   ├─ wheels.py      # 车轮（单轮 + 4 轮布局）
│   ├─ lights.py      # 大灯 + 尾灯
│   ├─ grille.py      # 进气格栅
│   ├─ mirrors.py     # 后视镜
│   ├─ seams.py       # 车门分缝线
│   └─ assembler.py   # 整车组装 + 统计 + 导出
├─
├─ freeform/          # 模块 2：NURBS 自由曲面内核
│   ├─ nurbs_core.py  # 曲线/曲面基函数与节点
│   ├─ freeform_surface.py # 自由曲面
│   ├─ swept_surface.py # 扫掠曲面
│   ├─ fillet_surface.py # 圆角曲面
│   └─ step_writer.py # 纯 Python STEP 写出
├─
├─ surface_quality/   # 模块 3：曲面质量评估 + AI 优化
│   ├─ curvature.py   # 曲率估算（法向 + 夹角）
│   ├─ continuity.py  # G0/G1/G2 连续性判定
│   ├─ reflection.py  # 反射线评分
│   └─ grader.py      # 综合等级（A/B/C/D）
```

```
|   └─ optimizer.py          # AI 模拟退火优化
|
|─ storyboard/              # 模块 4: 视频脚本生成
|   └─ scene.py             # 分镜 / Storyboard 数据结构
|   └─ templates.py         # 3 套内置模板
|   └─ generator.py         # 生成器（模板 + 自定义 + 时长缩放）
|
|─ storyboard_viewer/       # 模块 5: 视频脚本展示
|   └─ markdown_renderer.py # Markdown 渲染（表格/色卡/数据对比）
|   └─ html_renderer.py     # HTML 渲染（响应式 + 交互）
|
|─ tests/                   # pytest 200 测试（8 个测试文件）
|─ examples/                # 17 个完整使用示例
└─ outputs/                 # 默认输出目录（自检产物）
```

快速开始

1. 安装依赖

```
pip install -r requirements.txt
```

2. 一站式自检

```
python test_all.py
```

3. CLI 模式

构建默认参数整车

```
python main.py build-car
```

自定义参数 + 导出 GLB

```
python main.py build-car --L 4.8 --roof-arc 0.55 --output car.glb
```

评估球面

```
python main.py quality --shape sphere
```

AI 优化球面（80 次迭代）

```
python main.py optimize --shape sphere --iter 80
```

优化车身曲面

```
python main.py optimize --shape car --iter 80
```

生成视频脚本

```
python main.py storyboard --template car_promotion --duration 90
```

渲染为 HTML

```
python main.py render --format html --output story.html
```

跑完整流程

```
python main.py all
```

4. Python API 模式

```
from api import (
    build_car, get_car_stats, evaluate_surface, optimize_surface,
    make_storyboard, render_storyboard,
)
from car_modeling import CarParams, build_body
import numpy as np

# 1) 建模 (10 部件: body/hood/roof/windshield/rear_window + 4 轮)
params = CarParams(L=4.8, roof_arc=0.55)
parts = build_car(params)
print(f"整车: {len(parts)} 部件, 车身 {len(parts['body'].vertices)} 顶点")

# 2) 评估: 用参数化车身单侧网格 (build_full_car 的 body 为包围盒, 不可直接评估)
n_long, n_circ = 48, 24
mesh = build_body(params, n_long=n_long, n_circ=n_circ, process=False)
ring = 2 * (n_circ + 1)
grid = np.asarray(mesh.vertices).reshape(n_long + 1, ring, 3)
surface = grid[:, 0::2, :] # (49, 25, 3)
report = evaluate_surface(surface, "车身侧视")
print(f"等级: {report.grade}, 反射线: {report.reflection_score}")

# 3) 优化
result = optimize_surface(surface, "车身侧视", max_iter=80)
print(f"{result.initial_grade} → {result.final_grade}")

# 4) 视频脚本
sb = make_storyboard(template="car_promotion", duration=90)
print(f"分镜: {len(sb.scenes)} 个, 总时长 {sb.total_duration}s")

# 5) 渲染
md = render_storyboard(sb, "markdown")
html_doc = render_storyboard(sb, "html")
```

模块 1: 参数化车身建模

输入: 22 维 `CarParams` (车长 L / 车宽 W / 车高 H / 轴距 / 22 项参数)

算法:

- 车壳:** 沿车长方向取 N 个截面 (默认 48), 每个截面按"宽+高+形状"3 个函数生成轮廓;
`process=False` 保留截面环构造顺序
- 玻璃:** 4 个矩形 + 1 个天窗 (前/后挡风带角度、车顶天窗、左右侧窗)
- 车轮:** 单轮 = 轮胎 (圆柱) + 轮毂 (圆柱) + N 辐条 (长方体)
- 灯/格栅/镜/门缝:** 基于位置 + 尺寸的简单几何体
- 组装:** `build_full_car` 以包围盒为 body 快速装配 10 部件

输出: 10 个独立 `trimesh.Trimesh` (body/hood/roof/windshield/rear_window + wheel_fl/fr/rl/rr), 可单独操作或合并导出

模块 2: 曲面质量评估

核心概念:

- **G0:** 位置连续 (默认全部三角面满足)
- **G1:** 切线连续 (跨边法向夹角 $< 5^\circ$)
- **G2:** 曲率连续 (跨边法向夹角 $< 2^\circ$)

算法:

- 法向量估算:** 每个网格点的法向 = 周围三角形法向的面积加权平均
- 连续性判定:** 跨边法向夹角 $<$ 阈值 $\rightarrow$ 计入 G1/G2
- 反射线评分:** 曲率均匀度 (uniformity) $\times 0.5$ + 曲率平滑度 (smoothness) $\times 0.5$
- 综合等级:**
 - A 级: G2 比率 $> 85\%$ 且反射线 > 0.7
 - B 级: G2 比率 $> 70\%$ 且反射线 > 0.5
 - C 级: G2 比率 $> 50\%$
 - D 级: 其他

模块 3: AI 模拟退火优化

目标函数: 法向跳变平方和 (越小越光滑)

算法:

- 初始温度 $T = 1.0$
- 每步随机扰动一个内部点 (加高斯噪声 $\times T \times \text{lr}$)
- Metropolis 接受准则:**
 - 目标更优: 直接接受
 - 目标更差: 按概率 `exp(- $\Delta/T$ )` 接受 (允许暂时变差, 避免局部最优)

4. 降温: `T *= cooling` (默认 0.97)
5. 终止: `T < min_temp` 或达到 `max_iter`

输出: `OptimizationResult` (含优化前后对比 + 收敛曲线 + 最佳曲面)

模块 4: 视频脚本生成

输入: 产品名 + 时长 + 模板 + 卖点

算法:

1. 加载模板 (3 套内置: `car_promotion` / `tech_demo` / `minimal_showcase`)
2. 等比缩放每个分镜的时长, 使总和 = 目标 `duration`
3. 注入产品名到 `{product}` 占位符
4. 转为 `Storyboard` 对象

输出: 7-10 个分镜的 `Storyboard` 对象 (含场景/画面/字幕/对白/运镜/配色/数据)

模块 5: 视频脚本展示

Markdown 渲染:

- 一级标题 + 元信息
- 每个分镜一个二级标题
- 表格化镜头信息
- 数据对比表 (如果有 `data_highlights`)
- 配色色卡 (如果有 `color_palette`)

HTML 渲染:

- 响应式布局 (深色背景)
 - 卡片式分镜 (带 `badge` 标签)
 - 配色色卡预览
 - 数据对比高亮 (红色 `before` / 绿色 `after`)
 - 组件徽章
-

性能数据

模块	操作	耗时	输出
建模	10 部件整车（默认参数）	< 0.5s	176 顶点 / 316 面
评估	球面 (16×16)	< 0.3s	D 级 / G2 比率 0.199 / 反射线 0.291
评估	车身侧视 (49×25)	< 0.3s	C 级 / G2 比率 0.855 / 反射线 0.158
优化	模拟退火 80 步	1~3.5s	连续曲面 $\Delta G2=0$; 噪声平面反射线 0.330→0.339
脚本	7 镜生成	< 0.05s	90s 视频脚本
渲染	Markdown / HTML	< 0.05s	~2KB .md / ~9KB .html
全流程	run_full_pipeline	< 12s	建模+评估+优化+脚本+渲染

扩展指南

添加新部件

在 `car_modeling/` 下新建 `xxx.py`：

```
import trimesh
from .car_params import CarParams

def build_xxx(params: CarParams) -> trimesh.Trimesh:
    mesh = trimesh.creation.box(extents=[0.1, 0.1, 0.1])
    mesh.visual.face_colors = [100, 100, 100, 255]
    return mesh
```

在 `assembler.py` 的 `build_full_car` 中添加：

```
"xxx": build_xxx(params),
```

在 `__init__.py` 中导出即可。

添加新模板

在 `storyboard/templates.py` 的 `TEMPLATES` 字典中添加：

```
TEMPLATES["my_template"] = {
    "name": "我的模板",
    "scenes": [
        {"duration": 10, "name": "...", ...},
        ...
    ],
}
```

自定义优化器

实现 `surface_quality/optimizer.py` 的同接口：

```
def ai_optimize(surface_points, panel_name, **kwargs) -> OptimizationResult:
    ...
```

可替换为贝叶斯优化 / 遗传算法 / 强化学习等。

API 参考

`car_modeling.CarParams`

22 维整车参数（详见 `car_params.py`）。提供：

- `to_dict()`：转字典（含 length/width/height/waistline_ratio 别名）
- `validate()`：校验参数合法性，返回错误列表（空列表表示全部合法）

`api.build_car(params) -> dict[str, trimesh.Trimesh]`

返回 10 部件字典：`{body, hood, roof, windshield, rear_window, wheel_fl, wheel_fr, wheel_rl, wheel_rr}`

`api.get_car_stats(params) -> dict`

返回：`total_vertices, total_faces, components{name:{vertices,faces,color}}, bounds{min,max}, parts`（保留历史别名 `vertices, faces`）。

`api.evaluate_surface(points, panel_name) -> QualityReport`

返回：

```
QualityReport(  
    panel_name, grade, # A/B/C/D  
    g0_count, g1_count, g2_count,  
    g1_ratio, g2_ratio,  
    max_curvature_jump, # 度  
    mean_curvature,  
    reflection_score, # [0, 1]  
    details  
)
```

`api.optimize_surface(points, panel_name, max_iter, seed) -> OptimizationResult`

返回:

```
OptimizationResult(  
    initial_grade, final_grade,  
    initial_g2, final_g2,  
    initial_reflection, final_reflection,  
    iterations,  
    convergence_curve, # 目标函数值序列  
    best_surface # (N, M, 3) 最佳曲面  
)
```

`api.make_storyboard(...) -> Storyboard`

参数:

- `product_name` (str): 产品名
- `duration` (float): 目标时长 (秒)
- `style` (str): 视觉风格
- `key_features` (list): 核心卖点
- `audience` (str): 目标观众
- `template` (str): 模板名

`api.render_storyboard(storyboard, fmt) -> str`

- `fmt = "markdown"`: 返回 Markdown 字符串
 - `fmt = "html"`: 返回 HTML 字符串 (含内联 CSS)
-

示例

- `examples/example_1_full_car.py` - 完整汽车构建 + 导出
 - `examples/example_2_quality.py` - 质量评估 + AI 优化
 - `examples/example_3_storyboard.py` - 视频脚本生成 + 渲染
-

License

内部使用版本